Automotive Data Engine

Fixless Audits

Fixless Audits Fixless Audits improve the quality and accuracy of task auditing by allowing auditors to provide feedback without making any fixes. Fixless Audits are quicker and easier to create when compared to standard

Fixless Audits

Fixless Audits improve the quality and accuracy of task auditing by allowing auditors to provide feedback without making any fixes. Fixless Audits are quicker and easier to create when compared to standard audits where customers fix tasks.

These audits can be created through the API or in Scale's LidarLite auditing tool. Fixless Audits submitted via the API can be reviewed and adjusted in LidarLite, allowing auditors to add, delete, or edit Feedback Items as needed.

This guide walks through Fixless Audit creation and review using the API.

Create a Fixless Audit

Fixless Audits first require some basic information: the relevant task ID, audit result, and the audited task response URL. When you create a fixless audit via the API, you'll need to create an audit payload in the format outlined in this documentation.

Retrieve a Fixless Audit

Fixless audits can be retrieved via our endpoint, and are structured in the same format as when you create an audit. An example response format has been included below the Creation Payload below.

```
# Set up the headers for the request
headers = {
    "accept": "application/json"  # Specify that we want the response in JSON format
}

# Create an audit
body = {...} # FixlessAuditCreate
url = "https://api.scale.com/v1/audits"
response = requests.put(url, headers=headers, auth=(API_KEY, ''), json=body) # 201 resp
print(response.text)

# Get audits
url = "https://api.scale.com/v1/audits/?task_id=xxx&id=yyy" # provide task id or audit
response = requests.get(url, headers=headers, auth=(API_KEY, '')) # 200 response Fixles
print(response.text)
```

Feedback Items

Fixless Audits primarily consist of Feedback Items, which indicate errors, comments, confirmations, or flags on the task.

Fixless Audits can be submitted without any Feedback Items. This is a valid audit and indicates that the task has no errors.

Feedback Item Type (required)

We use `type` to indicate the purpose of the Feedback Item. The most common type is `error` which indicates a mistake on the task. Other types: `flag`, `confirmation`, `comment`, are less common and are not fixed by Scale or considered in quality score calculations.

- `Error`: **The annotation is incorrect**
- `Flag`: **The annotation needs to be reviewed by a customer auditor, does not get fixed by Scale, and does not affect quality scores.**

scale

Feedback Item Category (required)

`category` indicates the kind of issue found on the task.

- `missing` - indicates that an annotation is missing. This is also referred to as a false negative.
- `extraneous` - indicates that the annotation should not be present. This is also referred to as a false positive.
- `geometry` - indicates that the annotation has the wrong dimensions or shape
- `position` - indicates that the annotation is in the wrong place
- `attribute` - indicates that the annotation has an incorrect attribute property
- `label` - indicates that the annotation label (also referred to as class) is incorrect

Important: All error categories except for `missing` are specific to an `annotation_id`. `missing` errors require a `point` / `polygon` and a `missing_annotation_type` to indicate what type of annotation is missing and where it should be.

Feedback Item Scope (required)

`scope` indicates the location and duration of the issue. We have 4 different schemas for Feedback Item Scopes, which apply to different error categories. Details on typing and required properties can be found in the **Create Fixless Audit **section, below.

- `FeedbackItemScopeAnnotation` - indicates the annotation ID, frame interval, and attribute where the issue is present. This schema is used when the category is: `extraneous`, `geometry`, `position`, `attribute`, or `label`
- `FeedbackItemScopePolygon` - indicates the 3D or 2D position and frame interval where the issue is present. This schema is used when the category is: `missing`
- `FeedbackItemScopePoint` - indicates the 3D or 2D position and frame interval where the issue is present. This schema is used when the category is: `missing`
- `FeedbackItemScopeScene` - indicates that the issue is related to a scene attribute (not an individual annotation). This schema is used when the category is: `attribute`

threshold (e.g. cuboid position is off by <30cm). ** **These errors will not affect the quality score

- `Standard`: Used when the auditor wants to flag an error that exceeds our SLA error threshold (e.g. cuboid position is off by >30cm). These errors will affect the quality score
- `Severe` : Used when the auditor wants to flag an error that exceeds our SLA error threshold (e.g. cuboid position is off by >30cm) and the error is critical and requires special attention. These errors will affect the quality score

Feedback Item Description and Metadata (optional)

`description` is an open text field shown in the tasking and auditing UI. This field can be used to describe the error or fix.

`metadata` is an object that can be used to store data for internal tracking. For instance, in metadata you might store `metadata.is_verified_by_human: true` and `metadata.confidence_level: 0.83` if you were using an automated system to generate Feedback Items.

Feedback Item State

`state` does **not **need to be included in the Fixless Audit creation payload. Newly created Feedback Items are defaulted to `state: open` . Feedback Item states may change as Scale reviews and fixes tasks.

- `open` - default state. Indicates the issue has not been fixed yet
- `resolved` - indicates the issue has been fixed
- `disputed` - indicates that Scale disagrees with the issue
- `escalated` - indicates the customer disagrees with Scale's dispute of the issue
- `rejected` - terminal state indicating that the issue was incorrectly reported after review of the escalation

```
  comments?: string;
  target_response_url: string; // from task.response.annotations.url (task.response.ortl
  feedback_items?: FeedbackItemCreate[];
  metadata?: { [key: string]: any };
}

FeedbackItemCreate {
  type: 'comment' | 'error' | 'flag' | 'confirmation';
  scope: FeedbackItemScope;
  category:
    | 'attribute'
    | 'extraneous'
    | 'geometry'
    | 'label'
    | 'missing'
    | 'position';
  severity?: 'mild' | 'standard' | 'severe';
  description?: string;
  metadata?: { [key: string]: any };
}

FeedbackItemScope =
  | FeedbackItemScopeAnnotation
  | FeedbackItemScopePolygon
  | FeedbackItemScopePoint
  | FeedbackItemScopeScene;

// extraneous, geometry, label, position errors should use this scope
// most attribute errors should use this scope. The exception is scene attributes
FeedbackItemScopeAnnotation = {
  type: 'annotation';
  annotation_id: string;
  sensor_id?: string | number; // defaults to the scene's primary sensor (typically the
  attribute?: string; // attribute name. defined iff scoped on an annotation attribute
  interval: FeedbackItemTimestampRange;
}

// Only for missing annotation errors
```

```
  missing_annotation_class?: string; // label name for missing annotation
  sensor_id?: string; // indicate camera sensor id iff vertices are in camera coordinat
  interval: FeedbackItemTimestampRange;
}

// Only for missing annotation errors
FeedbackItemScopePoint {
  type: 'point';
  coordinates: [x, y];
  missing_annotation_type: AnnotationType; // type of missing annotation. see enum defi
  stationary: boolean; // only applies for cuboids and indicates if cuboid is stationary
  missing_annotation_class?: string; // label name for missing annotation
  sensor_id?: string; // indicate camera sensor id iff coordinates are in camera coordi
  interval: FeedbackItemTimestampRange;
}

// only used for scene attribute errors
FeedbackItemScopeScene {
  type: 'scene';
  attribute?: string; // scene attribute name
  interval: FeedbackItemTimestampRange;
}

FeedbackItemInterval = {
  type: 'frame_range';
  start: number; // frame index
  end: number; // frame index
};

// missing annotation types for FeedbackItemScopePoint and FeedbackItemScopePolygon
AnnotationType =
  // 3D - task types: sensorFusion, multiStage
  'cuboid' |

  // 2D - task types: videoAnnotation, multiStage
  'box_2d' |
  'polygon_2d' |
  'polyline_2d' |
```

```
  'polyline' |
  'point_topdown';
```

```
FixlessAudit {
  id: string;
  srn: 'srn:scale:avcv:audit:{{id}}' // SRNs can be used in place of id on endpoints
  type: 'fixless';
  result: 'accepted' | 'rejected';
  task_id: string;
  comments?: string;
  target_response: { url: string };
  feedback_items?: FeedbackItem[];
  metadata?: { [key: string]: any };
  active: boolean; // whether this is the latest audit
  source: 'api' | 'lidarlite' | 'classic';
  created_by: string;
  created_at: iso_date_string;
  updated_at: iso_date_string;
}
```

Calculating Quality Scores

Scale only uses Feedback Items of `type: error` and `severity: standard | severe` to compute quality scores.

Only the latest Fixless Audit is considered when computing task and batch quality scores.

Invalid Feedback Items are not considered. Invalid feedback items are marked in the `grader_output` property on the Feedback Item. Grader outputs are computed after the Fixless Audit is submitted and may not be available for a few minutes:

- `grader_output.conflict = true`
- `grader_output.explanation = "description of the issue..."`

- Attribute error where the attribute name does not match an attribute on the annotation
- Missing error where a point or polygon indicating the location of the error is not specified

Tasks Multi-Stage Task Reference